Field Hours

CSCE 709 Intermediate Program Design and Development with C++

Original notes for this site. Same topics as the catalog course. Not a reprint of another university's lectures, and not academic credit.

Lecture 1 · 90 minutes

Copies, references, and a small main

You already know statements and functions. A C++ program still starts in main and runs top to bottom. The declaration int n = 4 stores 4. void show(int x) copies 4 into x, so changing x does not change n. void bump(int &x) makes x another name for the caller's slot. bump(n) writes 5 back into n when n started at 4. You pass a copy when the function must not change the caller, and a reference when it must.

cout << n writes the stored 4. cin >> n replaces that slot with the next int from input. You check the stream after the read. If it failed, you stop instead of adding a value you do not have. A main that reads two ints, adds them, and writes the sum is the shell for this course. Later you hide a class, and the memory that class owns, behind that same main.

Copy versus another name.

Copy parameter

void show(int x)

x starts as a copy of 4

Assigning to x keeps n

Reference parameter

void bump(int &x)

x names the caller's slot

bump(n) stores 5 in n

Add 1 through 4 by reference

1. int s = 0. void add(int &s, int v) executes s = s + v.

2. Call add(s, 1), then add(s, 2), then add(s, 3), then add(s, 4).

3. The caller's slot holds 10. A copy parameter would have left that slot at 0.

Problem 1. n is 4. show takes int x and sets x = 9. bump takes int &x and sets x = 9. What is n after each call?

Answer. After show, n is still 4, because x was a copy. After bump, n is 9, because the reference names the caller's slot.

Problem 2. s starts at 0. add takes int &s and does s = s + v. You call it with 1, then 2, then 3, then 4. What is s, and what would the caller's s be if the parameter were a plain int copy?

Answer. s is 10. The partial sums are 1, then 3, then 6, then 10, because each call updates the same slot. A copy parameter would leave the caller's s at 0.

Lecture 2 · 90 minutes

const, overloads, and a returned sum

const on a reference is a promise the compiler checks. int total(const int &a, const int &b) may read a and b and may not assign to them. total(3, 5) returns 8. You use const when the job is to look, not to update. Drop const only on a parameter the body must change. A body that assigns through a const reference will not compile. That refusal is the mechanism. A comment is a weaker guard.

Two functions may share a name when the parameter types differ. int add(int a, int b) and double add(double a, double b) are overloads. add(2, 3) runs the int version and returns 5. add(2.5, 1.5) runs the double version and returns 4.0. The compiler picks from the argument types. It does not guess. If you pass two ints, you get the int body. Adding 7 and 2 that way returns 9.

One name, two bodies.

int add

Two int parameters

add(2, 3) returns 5

7 and 2 return 9

double add

Two double parameters

2.5 and 1.5 return 4.0

The fraction stays

A const parameter bounds a sum

1. int sum_to(const int &n) is called with n equal to 4. The body may read n and may not assign to it.

2. The loop runs i from 1 through n, adding i. The return is 10.

3. The caller's 4 is unchanged. You did not need a writable reference, because the job was to look at n and return a new int.

Problem 1. total takes two const int references and returns a + b. Can the body set a = 0 before returning, and what is total(3, 5)?

Answer. No. const forbids the assignment, so that body does not compile. total(3, 5) returns 8.

Problem 2. You have int add(int, int) and double add(double, double). What do add(2, 3) and add(2.5, 1.5) return?

Answer. add(2, 3) returns 5 from the int overload. add(2.5, 1.5) returns 4.0 from the double overload. The argument types pick the body.

Lecture 3 · 90 minutes

A class, a constructor, and one method

A class packs fields and the methods that use them. Each Rect holds two ints, w and h, plus a method area. Rect r(3, 4) runs the constructor, which copies 3 into w and 4 into h. r.area() returns 12. The call uses that object's fields. You put w and h in the private section so other code cannot write r.w. const on area promises not to change the two ints, and a write inside area will not compile.

Define the body as int Rect::area() const { return w * h; }. Rect:: names the class. You still call r.area(), not a free function. A free function would take the Rect as an extra argument. Keeping area on the class ties the two fields to the one operation. Callers that hold r do not rebuild the product. 3 times 4 is 12, and that product is the whole job of area. The constructor is where those fields are set.

One Rect, private fields.

Constructor stores w and h

private hides both ints

area is a const method

The call reads only that Rect

A 3 by 4 rectangle

1. Rect r(3, 4) stores w = 3 and h = 4 in r.

2. r.area() returns w * h, which is 12. const means the call leaves 3 and 4 in place.

3. Rect s(1, 1) has its own fields. s.area() returns 1. Calling it does not change r.

Problem 1. r is constructed as Rect(3, 4). What does r.area() return, and can that const method set w to 0?

Answer. It returns 12, because 3 * 4 = 12. It cannot set w to 0. const on the method rejects a write to the fields.

Problem 2. Why can main not assign r.w = 9 after Rect r(3, 4) if w is private?

Answer. private allows only Rect's own methods to name w. main must use the constructor or a method. The stored width stays 3 unless a method of Rect changes it.

Lecture 4 · 90 minutes

new in the constructor, delete in the destructor

A constructor runs when the object is created, and it is where you take a resource. Bag::Bag() sets n = 4 and data = new int[4]. Those 4 ints live on the heap, and data holds their address. The destructor ~Bag() runs when the Bag dies and executes delete[] data. That pair is the mechanism. If you allocate in the constructor and skip the

destructor, the 4 ints leak when the Bag leaves scope.

You do not call the destructor by hand for a local Bag. Leaving the block destroys it, and ~Bag() runs. new int[4] is matched with delete[], not with delete. The brackets say this is an array of 4, not one int. After delete[] you do not use data. A Rect of two plain ints needs no new and no delete. You add this resource only when the object owns memory the language will not free for you.

A Bag owns four ints.

1. Constructor runs new int[4]

2. Methods use the block

3. Scope ends, destructor runs

4. delete[] frees that block

Fill a Bag and let it die

1. Bag b; the constructor sets n = 4 and allocates four ints. Suppose you then store 1, 2, 3, and 4.

2. A method sum walks indexes 0, 1, 2, and 3 and returns 10. It does not allocate a second block.

3. When b leaves the block, ~Bag() runs delete[] data once. You do not also call delete on that address.

Problem 1. Bag::Bag() does new int[4], and the class has no destructor. What is wrong when a local Bag leaves its block?

Answer. The 4 ints stay allocated. Nobody calls delete[]. The constructor acquired a resource and nothing released it, so the block leaks.

Problem 2. The constructor used new int[4]. Which form belongs in the destructor, delete or delete[]?

Answer. delete[]. The brackets match an array new. delete without brackets is for a single object, not for the array of 4 ints.

Lecture 5 · 90 minutes

A copied pointer is a double free

If the compiler copies a Bag, it copies the pointer, not the 4 ints. Two Bags then hold one address. Each destructor runs delete[], so the second free gives that address back again. The fix is a copy you write: allocate a fresh int[4] and copy the four values. Each Bag then frees only its own block. You now have two heap arrays. The pointer copy is the mechanism that caused the double free.

You can forbid the copy when a Bag must stay unique. Deleting the copy constructor means a second Bag cannot be made from the first, so the shared pointer never appears. Pick one rule. Either each copy allocates its own 4 ints, or copying is not allowed. A warning in a comment is not a rule. If two objects exist, two destructors run, and each frees the pointer it holds.

Two ways to copy a Bag.

Compiler copy

Copies the pointer only

Both free one address

The second free is wrong

Copy you write

Allocates a new int[4]

Copies the four values

Each frees its own block

Hand a Bag to a function by value

1. b holds 1, 2, 3, 4 in a block from new int[4]. void show(Bag x) receives x by value.

2. The compiler copy makes x.data equal b.data. Both names are one address.

3. When show returns, ~Bag() runs for x and frees that address. When b dies, the second delete[] frees it again. A deep copy would have given x its own four ints.

Problem 1. Two Bags hold the same data pointer because one was copied from the other by the compiler. What happens when both destructors run?

Answer. Both call delete[] on one address. The first free returns the block. The second free returns that same address again, which is a double free.

Problem 2. You want show(Bag x) to be safe, and x should see the four values 1, 2, 3, and 4. What must the copy do?

Answer. Allocate a new int[4] and copy the four values into it. x then frees its own block, and the caller frees a different block. The sum of those values is still 10.

Lecture 6 · 90 minutes

A vector owns the array for you

A vector<int> grows a heap array and frees it in its destructor. Push 1, then 2, then 3, then 4, and size() is 4. You read indexes 0 through 3. Naming index 4 steps past those elements, and the bracket operator does not check. You do not write delete. The destructor does. A Bag you wrote by hand is the same 4 ints with the free left for you to get right.

A loop from i = 0 while i < size() adds each element into s. The sum of 1, 2, 3, and 4 is 10. size() is the bound. Passing the vector by value copies every element, all 4 of them. Passing a const reference looks at them and cannot write. You choose the reference when the function should not own a second array. For this sum, the const reference is the match.

Sum a vector of four.

Push 1, 2, 3, and 4 -> size() reports 4 -> Add indexes 0 through 3 -> Destructor frees the block

Return 10 without a manual delete

1. vector<int> v; push 1, then 2, then 3, then 4. v.size() is 4.

2. int total(const vector<int> &v) sets s = 0 and adds v[0] through v[3]. The return is 10. const blocks a write.

3. When v leaves scope, its destructor frees the heap array. total did not copy the four ints and does not delete them.

Problem 1. v holds 1, 2, 3, and 4. A loop adds v[i] while i < v.size(), and s starts at 0. What is s, and does your code call delete?

Answer. s is 10. You do not call delete. size() is 4, so the indexes are 0, 1, 2, and 3, and the vector destructor frees the block.

Problem 2. total only reads the vector. Why pass const vector<int> & instead of vector<int> by value?

Answer. A by-value parameter copies all 4 elements into a second array. The const reference looks at the caller's vector and cannot assign to it. The sum does not need a second owner.

Lecture 7 · 90 minutes

Private data and a checked index

An invariant is a fact you keep true after every public method. For a Bag, n is 4, data points at 4 ints, and no index at or past 4 is read. The constructor makes that true. get(i) returns data[i] only when i is at least 0 and less than n. When i is 4, you refuse. Callers cannot break the row just by asking. One checked method is the mechanism, instead of a test copied into every caller.

You test the invariant on a tiny bag, not with a speech. Store 1, 2, 3, and 4, and check that sum returns 10. Then call get(4) and check that it refuses. If the sum is 10 but get(4) still reads, the bound is missing. Public methods are the door. A public pointer lets a caller write index 4 and skip your test. Keep data private so the 4-slot rule stays in your code.

Check the index, then read.

1. Caller asks for index i

2. Reject i outside 0 to 3

3. Return data[i] if it fits

4. Sum of the four is 10

get(2) versus get(4)

1. The bag holds 1, 2, 3, 4, and n is 4. get(2) sees 2 < 4 and returns 3, the value at index 2.

2. get(4) sees 4 is not less than n. It refuses and does not read data[4].

3. sum still walks only 0, 1, 2, and 3 and returns 10. The refusal did not change the stored ints.

Problem 1. n is 4 and the slots are 1, 2, 3, 4. What does a checked get(2) return, and what does get(4) do?

Answer. get(2) returns 3, the int at index 2. get(4) refuses, because 4 is not less than n. It does not read past the last legal index, which is 3.

Problem 2. Why keep the data pointer private if get already checks the index?

Answer. A public pointer lets a caller write index 4 and skip get. Private data forces that write through your method, so the 4-slot rule still runs.

Lecture 8 · 90 minutes

A class that owns one resource

You can now build a small C++ type that owns one resource. You pick a copy or a reference at the call. You mark look-only parameters const. You write a class, a constructor that does new int[4], and a destructor that does delete[]. You either copy those 4 ints into a new block or you forbid the copy, so two objects never free one address. You can also say the invariant: private data, n slots, indexes below n.

Before you call the type done, run one object out of a block and see the destructor fire once. Store 1, 2, 3, and 4, and check the sum is 10. Then copy the object, or try to, and name the ownership rule you picked. If both destructors would free the same address, the copy is wrong. Fix the copy or forbid it. A class you cannot explain at 4 elements is not finished.

What you can now build.

A class with private fields

new in the constructor

delete[] in the destructor

A copy rule you can name

One Bag, summed, then destroyed

1. Bag b's constructor does new int[4] and you store 1, 2, 3, and 4. sum returns 10, reading indexes 0 through 3.

2. Pass b to int total(const Bag &x). The const reference does not copy the pointer and does not allocate.

3. When b leaves the block, one destructor runs delete[] once. There is no second owner, so nothing frees that address again.

Problem 1. A Bag is constructed with new int[4] holding 1, 2, 3, and 4. You pass it by value into a function that returns the sum, and both the parameter and the caller are later destroyed. What does the compiler's copy do, and what is the sum if the copy allocates its own four ints?

Answer. The compiler copies the pointer, so both destructors call delete[] on one address and the block is freed twice. A copy that allocates its own int[4] and copies the values returns 10, and each destructor frees only its own block.

Problem 2. total only reads a Bag. Should the parameter be Bag by value, or const Bag &? What does the sum of 1, 2, 3, and 4 equal either way, if ownership is safe?

Answer. Use const Bag &. By value copies the pointer unless you wrote a deep copy. The const reference does not allocate and cannot change the Bag. The sum is 10.